

P0 · CORALNPU 教学仓

P0 · 预备课：零基础桥接

作业说明 —— 任务拆解、验收标准与调试方法

对应上游仓库：google/coralnpu（RTL、文档、测试程序都来自这里）

学习方式：读规格 → 写 RTL → ./learn check 对拍 → 修复

文档版本：2026-09-13 · 生成命令：./learn pdf

目录

P0 作业说明（预备课）

0. 这门课怎么用

模块 A：命令行基础

模块 B：从 C 源码到机器码

模块 C：第一个硬件模块（组合逻辑）

模块 D：Verilog 的两类写法

模块 E：一条指令的字段拆解

验收与常见坑

P0 作业说明 (预备课)

0. 这门课怎么用

先做完 `./learn diag`，它会告诉你哪几层需要补。**每层都是独立的**，会的那层直接跳过，不要浪费时间。全部做完大约 1 小时。

五个模块与产物：

模块	主题	你要交的东西	预计时间
A	命令行：重定向、管道、退出码	<code>course/work/P0/A_terminal.txt</code>	5 分钟
B	从 C 到机器码：ELF、符号、反汇编	<code>course/work/P0/B_notes.txt</code>	15 分钟
C	数字逻辑：组合逻辑（多数表决器）	<code>course/lessons/P0_prep/rtl/majority.sv</code>	15 分钟
D	Verilog：多路选择器 + 计数器	<code>rtl/mux4.sv</code> 、 <code>rtl/counter.sv</code>	25 分钟
E	RISC-V：一条指令的字段拆解	<code>course/work/P0/E_journey.txt</code>	10 分钟

验收：

```
./learn check P0          # 全部
./learn check P0 --only a b  # 只跑某几个模块
./learn check P0 --ref      # 看参考实现能不能通过（课程自检）
```

模块 A：命令行基础

任务

在仓库根目录依次执行下面四条命令，把输出都存进一个文件：

```
mkdir -p course/work/P0
pwd > course/work/P0/A_terminal.txt
echo "hello-from-file" >> course/work/P0/A_terminal.txt
ls course/lessons >> course/work/P0/A_terminal.txt
false; echo "exit=$?" >> course/work/P0/A_terminal.txt
cat course/work/P0/A_terminal.txt      # 检查结果
```

期望看到的文件内容

```
/home/ubuntu/coralnpu
hello-from-file
L00_setup
L01_scalar
P0_prep
exit=1
```

（第一行是你自己的仓库路径，最后一行是 `false` 的退出码。）

为什么做这个

后面每节课的检查器、CI、脚本都在用这三件事：**重定向**（把输出留下来）、**追加**（`>>` 不覆盖）、**退出码**（判断成功失败）。这些是后面所有工作的基础。

模块 B：从 C 源码到机器码

任务 1：编译一个真实的裸机程序

```
python3 course/tools/build_program.py \
    course/lessons/L00_setup/tests/programs/hello.c \
    --out-dir course/work/P0/hello
```

它会生成 `hello.elf`、`program.hex`、`data.hex`、`symbols.json`。

任务 2：观察 ELF 里的东西

```
# 反汇编：看每条指令在哪个地址、机器码是什么
riscv64-unknown-elf-objdump -d course/work/P0/hello/hello.elf | head -20

# 符号表：看函数和全局变量被放在哪个地址
riscv64-unknown-elf-objdump -t course/work/P0/hello/hello.elf | grep -E "main$" | answer$"
```

任务 3：把三个值写进 `course/work/P0/B_notes.txt`

格式严格如下（等号两边不要有空格也可以，大小写不限）：

```
main=0x00000080
answer=0x00010000
first_sw=0x02812623
```

三个值分别是：

- `main`：`main` 函数在内存里的地址（从符号表或反汇编里找）；
- `answer`：`answer` 这个全局变量的地址（你会看到它在 `0x10000` 以上，也就是 DTCM）；
- `first_sw`：反汇编输出里**第一条** `sw` 指令的机器码（取那 8 位十六进制数）。

上面的数值是课程环境里编译出来的结果。你自己跑出来的应该一样（同样的源码、同样的工具链）； 如果不一样，以**你自己的输出**为准——检查器也会用它自己编译出来的 ELF 来核对。

模块 C：第一个硬件模块（组合逻辑）

任务

打开 `course/lessons/P0_prep/rtl/majority.sv`，实现三输入多数表决器：**a、b、c 里至少两个为 1 时，y 输出 1。**

文件里有真值表和一行 `assign y = 1'b0;` 的骨架，把 `1'b0` 换成你的表达式即可。

自检

```
./learn check P0 --only c
```

期望：40 组向量全部一致（8 组全枚举 + 32 组随机）。

提示（先自己想 10 分钟）

- 「至少两个为 1」= 「a 和 b 都为 1」或者「b 和 c 都为 1」或者「a 和 c 都为 1」；
- 与是 `&`，或是 `|`，异或是 `^`；
- 组合逻辑不要用时钟。

模块 D：Verilog 的两类写法

任务 1：mux4.sv（组合逻辑）

4 选 1 多路选择器：`sel=0` 选 `d0`，`sel=1` 选 `d1`，以此类推。骨架里已经写了 `always_comb`，你要补 `case` 的四个分支。

任务 2：counter.sv（时序逻辑）

带复位和使能的 8 位计数器：

- `rst=1` 时 `q` 清零（复位优先）；
- 每个时钟上升沿：`en=1` 时 `q` 加 1，`en=0` 时保持不变。

骨架里已经写了 `always_ff @(posedge clk)`，你要补 `if (rst) ... else if (en) ...`。

自检

```
./learn check P0 --only d
```

期望：`mux4: sel=0..3` 全部选择正确；`counter`：复位、10 次计数、保持都正确。

这两个练习为什么重要

- `mux4` 是处理器里出现最多的电路：寄存器堆读端口、写回数据选择、地址译码本质都是它；
- `counter` 是「时序逻辑 + 复位 + 使能」的最小模型：L01 的 PC、状态机、流水线寄存器写法和它一样。

模块 E：一条指令的字段拆解

任务 1：用标准工具查机器码

```
printf ' .text\n addi a0, a1, -7\n' > course/work/P0/e_case.S
riscv64-unknown-elf-as -march=rv32i -mabi=ilp32 -o course/work/P0/e_case.o course/work/P0/e_case.S
riscv64-unknown-elf-objdump -d course/work/P0/e_case.o
```

任务 2：填 `course/work/P0/E_journey.txt`

```
inst=addi a0, a1, -7
machine=0xff958513
rd=a0
rs1=a1
imm=-7
```

寄存器可以写 ABI 名字（`a0`）或编号（`x10 / 10`），检查器都认。

自检

```
./learn check P0 --only e
```

这道题的意义

你已经把一条指令从汇编追到了机器码，并且知道了它由哪些字段组成。L01 要做的，就是把这张表变成电路：字段 → 控制信号 → 数据通路动作。

验收与常见坑

全部验收

```
./learn check P0
```

期望：

```
[通过] 模块A 命令行
[通过] 模块B 编译流程
[通过] 模块C 组合逻辑（多数表决器）
[通过] 模块D Verilog (mux4 + counter)
[通过] 模块E 指令执行
```

5/5 个模块通过

常见坑

现象	原因	修法
command not found: riscv64-unknown-elf-gcc	工具链没装	<code>./learn doctor --install</code>
模块 A 说第一行不对	不在仓库根目录执行	<code>cd</code> 到仓库根目录再重跑四条命令
模块 B 说 <code>answer</code> 不对	抄错了地址，或抄成了别的符号	重新 <code>objdump -t ... grep answer</code>
模块 C 结果反了	写成了「至少一个为 1」（那是或）	三条「与」再「或」起来
模块 D 的 counter 差 1	复位优先写反了	<code>if (rst) q<=0; else if (en) q<=q+1;</code>
仿真输出里有 <code>x</code>	某个分支没赋值	给默认值，或补 <code>default</code>
<code>b_notes.txt</code> 找不到	文件名大小写不对	严格用 <code>B_notes.txt</code> / <code>A_terminal.txt</code> / <code>E_journey.txt</code>

卡住了怎么办

```
./learn hint P0 1    # 按顺序看提示
./learn unlock P0    # 对照参考实现（建议只在卡住时看）
```

做完 P0 之后，回到正式课程：

```
./learn start L00
```